Chapter 6: The context window and memory

When you have a conversation with a person, they remember what you said last week, last year, and sometimes decades ago. When you have a conversation with an LLM, it remembers only what is currently in the active conversation. The moment that conversation ends, the model forgets everything you discussed.

LLMs are 'stateless', meaning that they don't change as conversations occur. They don't know anything about the state of a conversation or anything else you told them. Instead, any short-term memory that LLMs exhibit comes from the 'context window': this is the LLM's entire memory of your conversation. Understanding how it works is essential for using these tools effectively.

6.1 The context window

Here is how this works in practice. When you open a new chat, the context window is empty. You type a question, say, 'What is the capital of Texas?' That question is the entire context, and the model reads it and replies, 'Austin.'

Now you ask a follow-up: 'What is its population?' The LLM has no memory of your first question. If you just sent this question about population to the LLM, it would have no idea what 'its' is referring to. To fix this, the chat application sends the entire conversation to the LLM: your first question ('What is the capital of Texas?'), its own answer ('Austin'), and your new question ('What is its population?'). Because the earlier exchange is included, the model can work out that 'its' means Austin's.

Every subsequent question you ask and answer the LLM gives adds to the pile, and the whole conversation is sent back to the model each time. This is how an LLM can carry on a long conversation with you: it is not remembering anything, it is rereading the entire conversation and answering based on that.

When you close a chat window and open a new one, the context is reset and the model has no memory of any previous chat. This new conversation starts from zero. Some users find

Copyright © 2026 Andrew Dessler

this disorienting — they expect the AI to 'remember' them the way a human assistant would.

On the other hand, starting a new conversation is the right move when you change subjects. Everything said earlier in a chat is still in the context window, and the model may try to use it, even when it no longer applies. Suppose you spend an hour with an LLM polishing your résumé, and then, in the same chat, you ask it to help with a friend's résumé. Your own résumé is still in the context window, so the model may blend the two, giving your friend a degree they never earned or a job they never held. It is not being careless; as far as the model can tell, everything in the context window is relevant. The lesson is: Whenever you move to a new subject or task, start a new conversation.

6.2 Scale: the evolution of context windows

How big the context window is sets how much the model can remember about a conversation. Early LLMs in the late 2010s were limited to 512 to 1,024 tokens — barely enough for a few paragraphs of text (one token is roughly 0.75 words). GPT-2 and GPT-3 pushed this to 1,024 and 2,048 tokens respectively. GPT-3.5, which powered the original ChatGPT, offered 4,096 tokens — about 3,000 words.

By 2023, Anthropic introduced Claude with a 100,000-token context window (75,000 words) — enough to ingest a short novel. This enabled use cases that were previously impossible: analyzing entire legal contracts, reviewing large codebases, processing lengthy research papers. By 2024, Google's Gemini 1.5 Pro offered one million tokens, and today's frontier models routinely offer context windows of a million tokens. Meta's Llama 4 features a context window of ten million tokens — enough to hold an entire library of books.

6.3 When the context window fills up

Even a very large context window can fill up, particularly if you are pasting large documents into the chat. In a long conversation, every message you send, every document you paste in, and every reply from the model adds to the total, and eventually the limit can be reached.

Copyright © 2026 Andrew Dessler

When that happens, the software has to make room. One way to handle the limit is truncation, which means leaving out some information. For example, the software might remove the oldest messages while keeping newer ones. Another method is compaction, which means replacing earlier messages with a shorter summary. This summary tries to preserve important facts, decisions, and instructions. However, it does not necessarily preserve every detail. If a number or condition is left out, the model may not have that information when answering a later question.

This has practical impacts. If you give the model important instructions, those can be erased when the context window fills up. For example, in February 2026, Business Insider reported that an employee of Meta's Superintelligence Labs had given an AI agent called OpenClaw access to her email. Her instruction was clear: look through the inbox, suggest what to archive or delete, and don't act until she gave the go-ahead.

Her real inbox was large and, as the agent added emails to the context window in order to evaluate them, the conversation eventually exceeded the size of the context window. When it was compacted, the summary left out her instruction not to delete messages without approval. With that constraint gone, the agent began trashing her emails.

The instruction was not overridden or ignored. It was simply no longer in the context window, so as far as the model was concerned, it had never been given. Anything that must never be forgotten, especially a rule about actions that cannot be undone, should not depend on a single message in a long conversation.

6.4 A more persistent memory

I said above that, when you start a conversation, the context window starts out empty. That's not always right; let me explain why.

Most frontier LLMs have memory features that allow an LLM to 'remember' information about you over multiple conversations. Figure 6.1 shows the controls in ChatGPT. When memory is turned on, ChatGPT notes facts about you that it obtains from analyzing your previous conversations with it. For example, if I send it a lot of messages about climate change, it figures out that I work in that area.

Copyright © 2026 Andrew Dessler

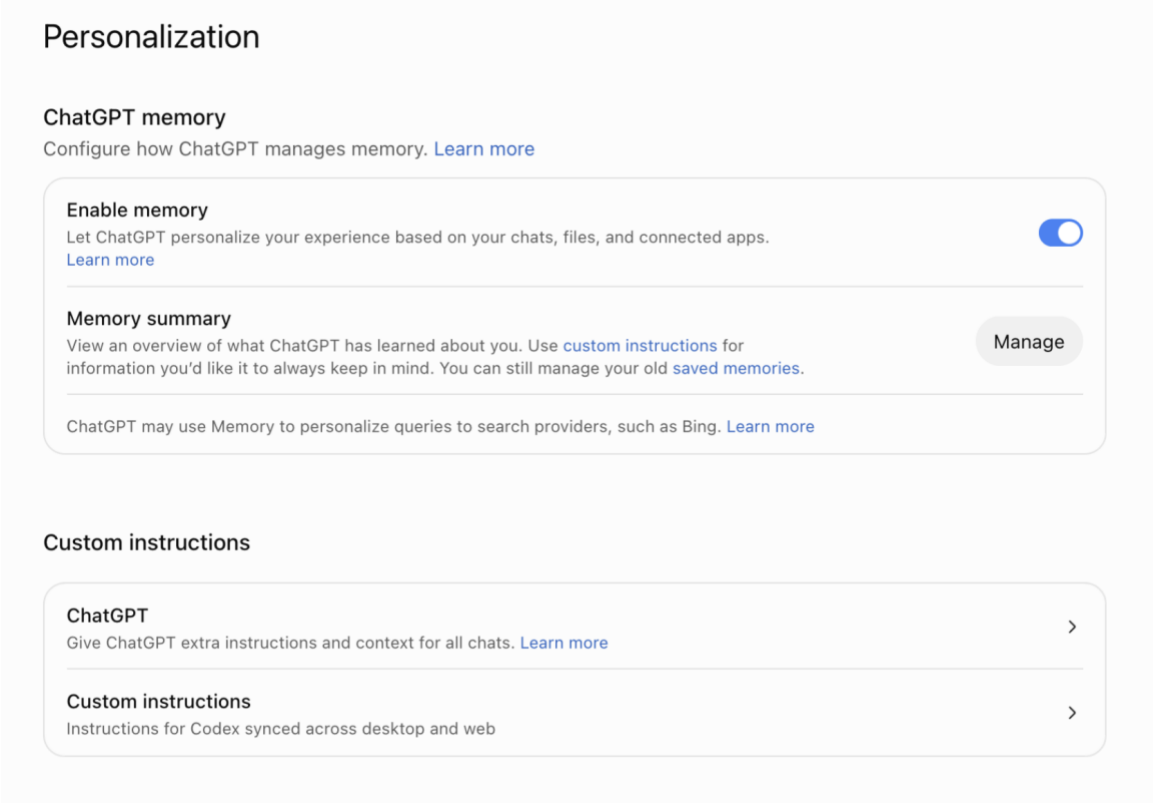


Figure 6.1. ChatGPT's personalization settings. Memory is switched on, and custom instructions appear below it. Source: screenshot of ChatGPT (OpenAI).

Figure 6.2 shows an example: the memory summary that ChatGPT built about me. I never wrote any of it. ChatGPT composed it on its own from my past conversations, and it is remarkably detailed. It describes my job, my research projects, and other details of my professional life, and it even records how I like things explained. When I start a new chat, ChatGPT injects this summary into the context window, which allows it to tailor responses.

While it's generally correct, there are some errors in it. For example, I do ask LLMs a lot of questions about AI, programming, etc., but I'm not taking courses in it. So, as with all LLMs, the memory can contain hallucinations.

Copyright © 2026 Andrew Dessler

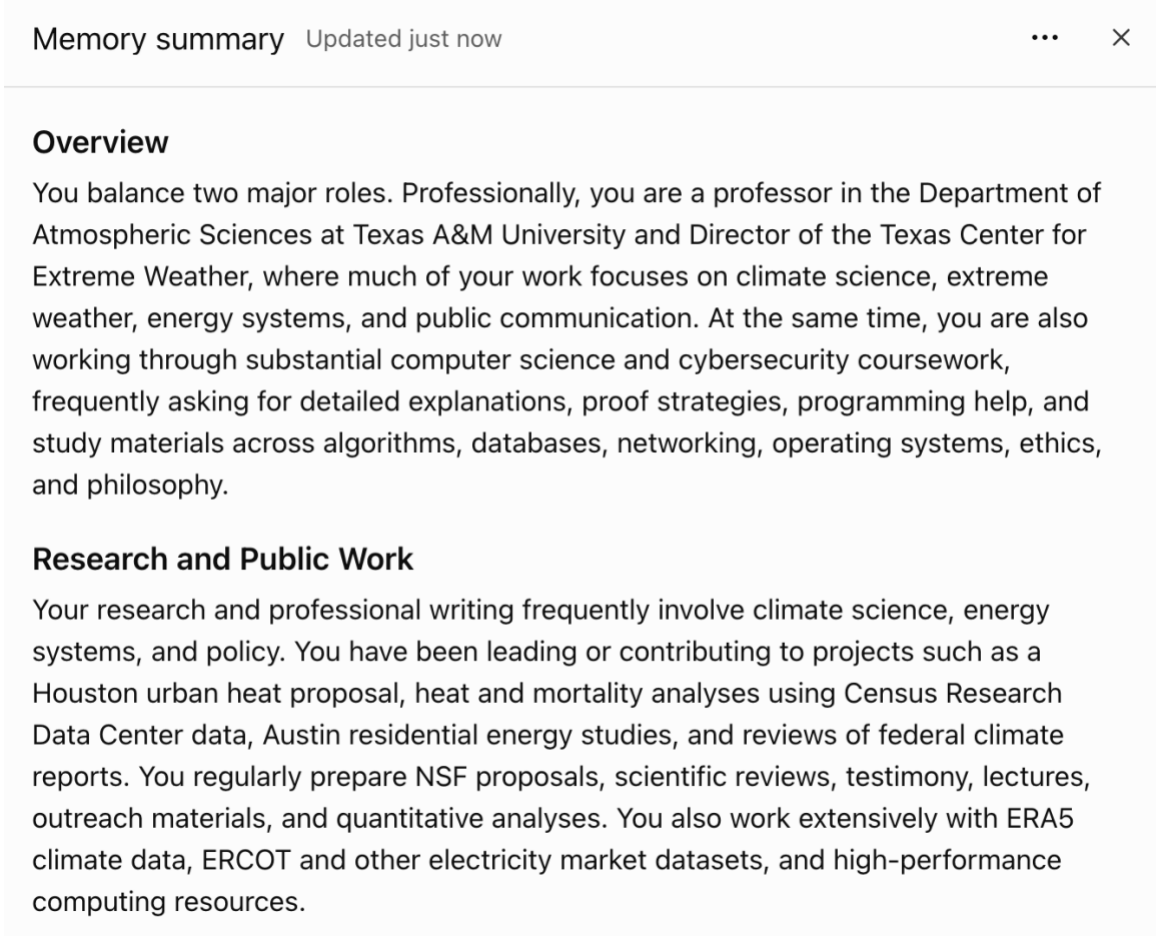


Figure 6.2. A memory summary that ChatGPT wrote about the author from past conversations. Only the beginning is shown. Source: screenshot of ChatGPT (OpenAI).

When I start a new conversation, ChatGPT adds this summary to the context window before it reads my first message. When it answers my question, it can then work in details based on my interests. For example, if I ask it to explain how a hash table works, it can pitch the explanation to a scientist who programs rather than to a computer science major, walk through every step because it knows I like to see the intermediate steps, and illustrate the idea with a climate dataset instead of a list of customers. I did not have to say any of that in my question; it was already in the context window.

A second feature is custom instructions. Unlike a memory, which the model writes for itself, custom instructions are written by you. The frontier labs all have the ability to give the LLM certain facts or instructions that are included in every chat. For example, you could tell it that you're a student at Texas A&M, your major, and that you want explanations without jargon.

Copyright © 2026 Andrew Dessler

When you start a new chat, the platform adds your custom instructions (and the memory summary described above) to the context window, so the model reads them before it reads anything you type. Like the memory summary, custom instructions are therefore not true memory: they are simply more text in the context window.

You can create memories yourself. Imagine, for example, that you want to keep a move a conversation from ChatGPT to Claude. You could ask ChatGPT to summarize the conversation and put it in a text file that you download — this is the equivalent of creating a memory of this conversation. You can then upload the file into Claude (thereby adding it to the context window) and continue the conversation there.

6.5 Key takeaways

- The context window is an LLM's working memory — it contains everything the model 'knows' about your conversation. In a chat, the whole conversation is resent with every new message.
- Context windows have grown substantially over time. Modern models typically have a context window of a million tokens, enabling use cases like full-book analysis.
- When a conversation outgrows the window, the system drops old messages (truncation) or replaces them with a summary (compaction), and important instructions can be lost along the way.
- When you reach the end of a conversation, you should start a new conversation so you start the next chat with an empty context window.
- LLMs have no persistent memory: every new conversation starts from scratch. Memory features and custom instructions are workarounds that place saved information into the context at the start of a conversation; they are not true recall.

Copyright © 2026 Andrew Dessler